



COMPUTABILITY AND COMPLEXITY

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

Major Objectives

- Today's Learning!
 - **Complexity Classes**
 - **Types of Complexity Classes**

What is Complexity Class?

- ✓ In computer science, some problems exist whose solutions are not yet found; the problems are divided into **Complexity Classes**.
- ✓ These classes help group problems based on **how much time and space they require to solve problems** and **verify the solutions**.
- ✓ The common resources are time and space, meaning **how long the algorithm takes to solve a problem and the corresponding memory usage**.
- ✓ The **time complexity** of an algorithm is used to describe
 - ✓ the **number of steps** required to solve a problem,
 - ✓ **how long it takes to verify** the answer.
- ✓ The **space complexity** of an algorithm describes
 - ✓ How much **memory is required** for the algorithm to operate?

Types of Complexity Classes

✓ Complexity classes are useful in organizing similar types of problems.

✓ Types of Complexity Classes

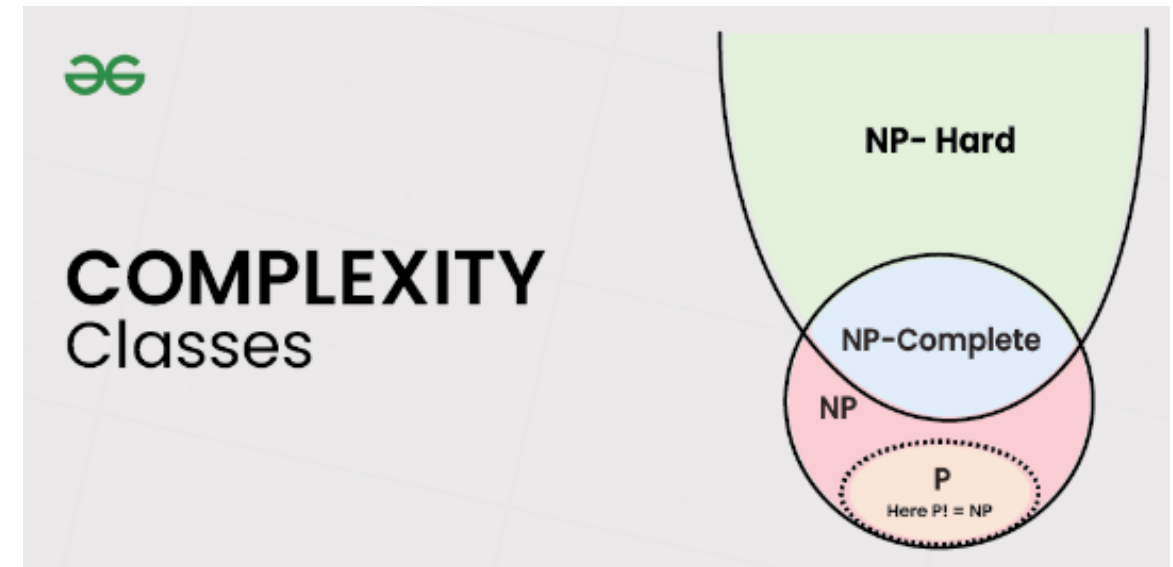
✓ P Class (Polynomial Time)

✓ NP Class (Non-deterministic Polynomial Time)

✓ Co-NP Class (complement of NP class)

✓ NP-hard Class

✓ NP-Complete Class



P Class (Polynomial Time) (Easy to Solve & Verify)

- ✓ The P in the P class stands for **Polynomial Time**. It is the collection of decision problems (problems with a “yes” or “no” answer) that can be **solved by a deterministic machine in polynomial time**.
- ✓ **Features:**
 - ✓ The solution to P problems is easy to find.
 - ✓ P is often a class of computational problems that are **solvable and tractable**.
 - ✓ Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.
- ✓ This class contains many natural problems like:
 - ✓ **Simple arithmetic** (e.g., What is 453×7 ?)
 - ✓ **Sorting a small list** (e.g., Sort {8, 3, 10, 1} in ascending order.)
 - ✓ **Finding the shortest path in a small graph** (e.g., What is the shortest route between A and B in a given simple map?)
 - ✓ **Checking if a number is even** (e.g., Is 12345678 even?)
 - ✓ **Merge Sort**

P Class (Polynomial Time)

(Easy to Solve & Verify)

Definition:

Problems that can be **solved efficiently** (in polynomial time).

If you can compute the answer in “reasonable time” even for large inputs — it’s in **P**.

Real-life examples:

Scenario	Problem	Why It's in P
 Sorting products in a warehouse	Sort by price or weight (MergeSort, QuickSort)	Solvable in $O(n \log n)$ time
 GPS navigation	Shortest route with Dijkstra's algorithm	Polynomial-time algorithm
 Database search	Binary search in a sorted table	$O(\log n)$ steps
 Spreadsheet calculations	Multiplying numbers, computing sums	Direct formula evaluation



Summary:

$P = \text{“Easy to solve and easy to verify.”}$

NP Class

(Non-Deterministic Polynomial Time)

(Hard to Solve, Easy to Verify)

- ✓ The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a **non-deterministic machine in polynomial time**.
- ✓ **Non-deterministic polynomial time (NP)** is a complexity class used to classify decision problems. If a problem is in non-deterministic polynomial time, the non-deterministic Turing machine can first guess at the solution, and then run a verifiable algorithm that will confirm whether or not that guess was correct.
- ✓ A decision problem is said to be in NP if it is solvable in polynomial time by a non-deterministic Turing machine. The polynomial time part means that if the non-deterministic algorithm makes all the right guesses, then the amount of time it takes is bounded by a polynomial.

NP Class

(Non-Deterministic Polynomial Time)

- **Examples:**

- **Sudoku puzzle (9x9 grid with missing numbers)** – Hard to solve but easy to check if a completed solution is correct.
- **Subset Sum Problem** (Find a subset of {3, 7, 11, 15} that adds up to 18.)
- **Traveling Salesman Problem (TSP)** (Find the shortest route through multiple cities with given distances.)
 - You will struggle to find the correct solution but, once given an answer, can check it quickly.



- **Summary:**

NP = “*Hard to find, easy to check.*”

P vs NP in COVID-19 Vaccination

✅ P-Class (Easy to Solve & Check)

1. **Checking if someone is fully vaccinated** → Just look at their record!
2. **Finding the nearest vaccine center** → Use Google Maps.
3. **Scheduling vaccine appointments** → Assign time slots like booking a ticket.
4. **Tracking vaccine inventory** → Simple counting: How many left?

🧐 Key Takeaway

- **P Problems** → Easy to solve & check.
- **NP Problems** → Hard to solve but easy to check once solved.
- If **P = NP**, we could **instantly** find the best way to stop pandemics! 🚀

🔴 NP-Class (Hard to Solve, Easy to Check)

1. **Best way to distribute vaccines to many cities?** 🤖
 - Too many possibilities → Need trial & error.
 - If given a plan, checking if it works is easy.
2. **Predicting future outbreaks** 🧠
 - Hard to solve because many factors change.
 - Easy to verify if a prediction comes true.
3. **Finding the best vaccination strategy**
 - By age? By region? By job? Too many choices!
 - Easy to check if one strategy worked.

Co-NP Class (Complement of NP) (Hard to Verify, Easy to Reject)

✓ Problems where **checking a solution is hard**, but **rejecting a wrong one is easy**.

✓ **Example: Verifying a Bad Vaccination Plan**

✓ **Question:** Is this vaccination plan **not** effective?

✓ If someone claims the plan is ineffective, we must check all possible cases to confirm.

✓ **Hard to verify** because we must analyze multiple factors (infection rates, immunity, hospitalizations).

✓ **Easy to reject** if we find just one case where it worked.

✓ **Conclusion: Hard to verify a solution but easy to disprove a bad one.**

NP-complete Class

Hard to find a solution, but easy to verify once given one.

- ✓ Intuitively NP-complete means that we can solve Y quickly if we know how to solve X quickly.
- ✓ **Features:**
 - ✓ If one could solve an NP-complete problem in polynomial time, then one could also solve any NP problem in polynomial time.
 - ✓ **Example:** If Pfizer get the vaccination of Covid, then other pharmaceutical companies could come up with their own variants like Moderna.

The **hardest problems in NP**.

If you solve one NP-Complete problem efficiently, *all NP problems become easy*.

NP-Complete = *“Hardest of NP problems — both in NP and NP-Hard.”*

NP-hard Class

(Hard to Solve, long-time to Verify)

✓ Problems that are **at least as hard as NP-Complete problems**, but may not even be in NP (they might not have efficiently verifiable solutions).

✓ **Features:**

✓ All NP-hard problems are not in NP.

✓ It takes a **long time to check** them because they do not have to be decision problems. This means if a solution for an NP-hard problem is given, it takes a long time to check whether it is right or not.

NP-Hard = “At least as hard as NP-Complete — possibly even impossible to verify efficiently.”

NP-Hard Example: Minimizing Vaccine Wastage

Problem: How to Distribute Vaccines to Avoid Wastage?

- COVID-19 vaccines have **expiration times** and must be stored properly.
- Each vial contains **multiple doses** (e.g., Pfizer = 6 doses per vial).
- If not used in time, **doses go to waste**.
- Goal: **Minimize wastage while ensuring fast distribution**.

Why is it NP-Hard?

- Similar to the **Bin Packing Problem** (efficiently grouping doses into appointments).
- Too many factors:
 - Number of doses per vial
 - No-show appointments
 - Storage limitations
 - Distance to vaccination centers
- Finding the absolute best way to distribute vaccines **requires testing all possibilities**, which grows exponentially.

Why is it Easy to Check a Solution?

- Given a vaccine distribution plan, we can **check the total wastage** by counting leftover doses.

Key Takeaway

- Finding the best way to distribute vaccines with zero wastage = NP-Hard.
- Checking if a given plan wasted doses = NP (Easy to Verify).

Summary

Complexity Class	Characteristic feature	Real-Life Analogy
P	Easily solvable in polynomial time.	Sorting a contact list
NP	Correct answers can be checked in polynomial time.	Solving a Sudoku
Co-NP	Incorrect answers can be checked in polynomial time.	Proving a Sudoku has no solution
NP-complete	A hardest problem that is in NP	Finding the best delivery route
NP-hard	A hardest problem in NP and NP-Complete and it takes a very long time to check them.	Predicting chess outcomes or halting behaviour